

Python Requests Library — Methods and Use Cases Guide

This guide summarizes the most commonly used methods, classes, parameters, and utilities from the Python **requests** library, along with practical guidance on when to use each.

Method / Feature	Purpose	When to Use	Example
requests.get()	Send an HTTP GET request	Use when retrieving data from APIs or webpages without modifying state	requests.get(url)
requests.post()	Send an HTTP POST request	Use when submitting forms, creating resources, or sending JSON payloads	requests.post(url, json=data)
requests.put()	Replace or update a resource	Use for full updates to an existing API resource	requests.put(url, json=data)
requests.patch()	Partially update a resource	Use when modifying only selected fields in a resource	requests.patch(url, json=data)
requests.delete()	Delete a resource	Use for removing records or API resources	requests.delete(url)
requests.head()	Retrieve headers only	Use when checking metadata, file existence, or response headers without loading content	requests.head(url)
requests.options()	Check supported HTTP methods	Use when discovering server/API capabilities	requests.options(url)
requests.request()	Generic request interface	Use when dynamically choosing HTTP methods in reusable code	requests.request(method, url)
response.json()	Parse JSON response	Use when API responses are JSON formatted	response.json()
response.text	Access response body as text	Use when reading HTML, XML, or plain text	response.text
response.content	Access raw binary response	Use when downloading files or images	response.content
response.status_code	Read HTTP status code	Use for validation and error handling	response.status_code
response.headers	Access response headers	Use for metadata like content type or authentication info	response.headers
response.raise_for_status()	Raise exception for HTTP errors	Use for automatic error handling in production code	response.raise_for_status()
requests.Session()	Persist settings and cookies	Use for multiple requests to the same site or authenticated sessions	session = requests.Session()
session.get()	GET request through session	Use when maintaining cookies/authentication across requests	session.get(url)
requests.cookies	Handle cookies	Use for login systems or tracking session state	response.cookies
requests.auth.HTTPBasicAuth	Basic authentication helper	Use when APIs require username/password authentication	HTTPBasicAuth(user, pass)
headers parameter	Send custom HTTP headers	Use for authorization tokens, content types, or user agents	headers={'Authorization': 'Bearer TOKEN'}
params parameter	Add URL query parameters	Use for filtering/searching API requests	params={'page':1}
json parameter	Automatically send JSON payload	Use when APIs expect JSON body content	json={'name':'Alice'}
data parameter	Send form-encoded payloads	Use for HTML forms or legacy APIs	data={'username':'bob'}

Method / Feature	Purpose	When to Use	Example
files parameter	Upload files	Use when sending documents/images to servers	files={'file': open('a.txt','rb')}
timeout parameter	Set request timeout	Use in production to avoid hanging requests	timeout=5
allow_redirects	Control redirects	Use when debugging redirect behavior or preventing automatic redirects	allow_redirects=False
stream=True	Stream large responses	Use for large downloads to reduce memory usage	requests.get(url, stream=True)
verify parameter	Enable/disable SSL verification	Use for secure HTTPS validation or testing environments	verify=True
cert parameter	Provide SSL client certificate	Use for mutual TLS authentication	cert=('cert.pem','key.pem')
proxies parameter	Route traffic through proxies	Use for debugging, scraping, or network routing	proxies={'http':'http://proxy'}
hooks parameter	Attach callback hooks	Use for logging or modifying responses automatically	hooks={'response': callback}
requests.exceptions	Exception handling classes	Use for reliable network error handling	except requests.Timeout:
requests.Timeout	Timeout exception	Use when handling slow network conditions	except requests.Timeout:
requests.ConnectionError	Connection failure exception	Use when remote server/network is unreachable	except requests.ConnectionError:
requests.HTTPError	HTTP error exception	Use when handling invalid HTTP responses	except requests.HTTPError:
response.iter_content()	Iterate over streamed content	Use for chunked file downloads	for chunk in response.iter_content(1024):
response.iter_lines()	Iterate over streamed lines	Use for streaming APIs or logs	response.iter_lines()
PreparedRequest	Prepare request before sending	Use for advanced customization/debugging	req.prepare()
Session.mount()	Attach custom transport adapters	Use for retries or protocol customization	session.mount('https://', adapter)
HTTPAdapter	Transport adapter with retries	Use for resilient API communication	HTTPAdapter(max_retries=3)

Tip: In production applications, combine timeout handling, sessions, retries, and structured exception handling to build reliable API clients.